

Tensor Monte Carlo, Massively-Parallel RWS, and the Source Term Trick

How alan does inference — plus a look at some newer empirical follow-ups

The problem

Tensor Monte Carlo

Massively-Parallel RWS

The MP-IW bound

The source term trick

How it fits together

Newer empirical follow-ups

The problem

Why one sample per latent isn't enough

- Standard importance sampling / VI: draw *one* sample of the whole latent vector z from Q , reweight by $p(z, x)/q(z)$.
- Better: draw K *whole-vector* samples (IWAE-style) and average — tighter bound, lower variance, but cost is $O(K)$ per *full model*, and you need K large before it helps on models with many latents.
- What if we could afford K samples *per latent variable*, not per whole vector?
Naively that's $K^{\#\text{latents}}$ combinations to consider — looks hopeless.

This is the problem Tensor Monte Carlo, then Massively-Parallel RWS, and then the source term trick, solve in turn.

Wake-Sleep, briefly

Classic wake-sleep (Hinton et al.) trains a generative model P and a recognition/proposal model Q together:



Reweighted Wake-Sleep (RWS) improves the Q update: instead of training Q on P 's own samples, use self-normalized importance weights from K samples of Q to push Q toward the *true posterior* directly.

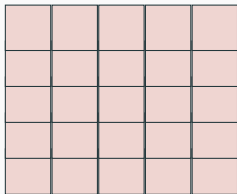
Tensor Monte Carlo

The key move that makes “ K samples per latent” tractable:

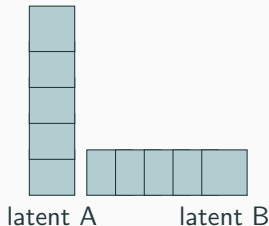
- Draw K candidate values for *each* latent variable separately, not K whole-vector samples.
- Most latents only interact with a handful of neighbours in the model’s dependency graph (plates, conditional independence).
- So the $K^{\#\text{latents}}$ sum factorizes into a sequence of small tensor contractions (like einsum) that follow the graph structure — cost becomes polynomial in K , not exponential.

Why this matters

naive: all K^2 combinations



structured: contract one latent at a time



Same K , same model — but reducing latent by latent (following the graph) instead of enumerating the full product turns an exponential sum into a polynomial one.

What TMC gives you — and doesn't

- + A much lower-variance, K -per-latent marginal likelihood estimator, at polynomial cost.
- On its own, it's just an estimator — no prescription for *training* P and Q together, and no cheap way to get posterior samples/moments back out afterward.

Massively-Parallel Reweighted Wake-Sleep (Heap et al., UAI 2023) turns this estimator into a full training algorithm.

Massively-Parallel RWS

The estimator: $P_{\text{MP}}(z)$

For each latent, draw K particles from Q ; contract following the plate structure (as above) to get a single scalar:

$$\hat{P}_{\text{MP}}(x) = (\text{TMC-style contraction of } p(z, x)/q(z) \text{ over all latents' } K \text{ particles})$$

- **Unbiased:** $\mathbb{E}[\hat{P}_{\text{MP}}(x)] = p(x)$, the true marginal likelihood — proved directly, not just assumed.
- This single fact is what later makes *exact* MCMC schemes (Particle MCMC) possible on top of this machinery — more on that at the end.

Two updates, one estimator

Wake phase (update P)

Maximize $\log \hat{P}_{\text{MP}}(x)$ — the massively-parallel analogue of the IWAE bound (more on this next).

Sleep phase (update Q)

Self-normalize the same per-latent importance weights, use them to push Q toward the true posterior — the “reweighted” part of RWS, now done per-latent instead of per-whole-vector.

Both updates reuse *the same* K -particle contraction — no separate machinery for training P vs. Q .

The MP-IW bound

From IWAE to Massively-Parallel Importance-Weighted

- The IWAE bound: average K *whole-vector* importance weights, take the log — gets tighter (closer to $\log p(x)$) as K grows.
- **MP-IW**: the same idea, but the K particles are per latent, contracted via the plate structure — so the bound gets tighter per unit of *actual* compute, not just per unit of K .
- Practically: many more effective “combinations” are being averaged over for the same memory/compute budget.

This bound is exactly what `alan` exposes as `sample.elbo_rws()` / `elbo_vi()` in the code.

The source term trick

The remaining problem

MP-RWS gives you a great scalar (the bound) and gradients for training.

But once you have a trained P , Q : how do you get

- posterior **samples**,
- posterior **marginals** (the per-latent weight distribution over its K particles),
- posterior **moments** (means, variances, covariances)

back out, without writing a bespoke resampling algorithm for every different plate topology?

The trick

Using Autodiff to Estimate Posterior Moments, Marginals and Samples (Bowyer, Heap & Aitchison, UAI 2024):

1. Add an auxiliary **source term** J into the log joint, additively, before running the *exact same* K -dim contraction used to compute the bound.
2. That contraction now computes a scalar $L(J)$ instead of just $L(0)$.
3. Differentiate $L(J)$ w.r.t. J , at $J = 0$, using ordinary autodiff. The derivative is the marginal / moment you wanted.

Exactly the same move as getting moments from $\partial \log Z[J] / \partial J$ for a log-partition function with an external source J — a very old idea in statistical physics and exponential families, applied here to a Monte Carlo estimator instead of an analytic one.

Why this is the elegant part

- No new algorithm per quantity — `alan` computes posterior samples, marginals, *and* moments through literally the same code path that computes the bound.
- Works uniformly across arbitrarily nested plates — the contraction already handles that; the source term rides along for free.
- It's autodiff doing the bookkeeping that would otherwise be a hand-written, model-specific resampling routine.

This is really in the code

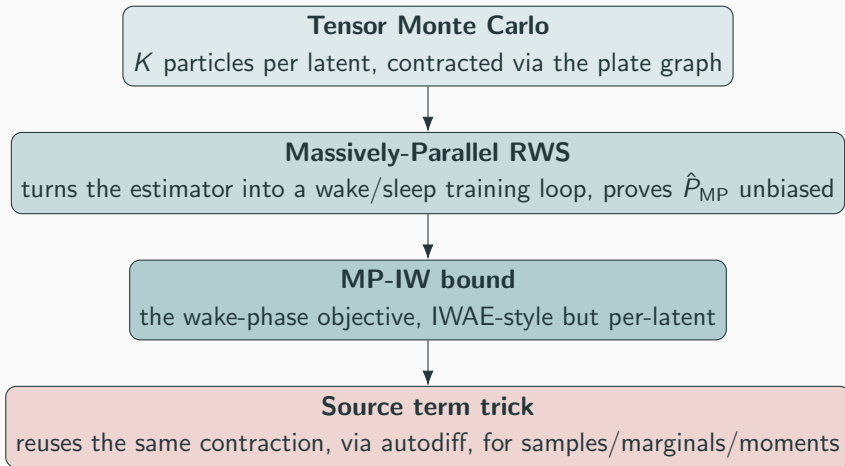
From alan's `Sample.py` (lightly trimmed):

```
J_tensor = zeros(shape, requires_grad=True)
L = self._elbo(sample, extra_log_factors={...: J_tensor})
marginals = grad(L, J_tensor_list)
```

- `J_tensor` starts at all zeros — it doesn't change what `L` evaluates to.
- But differentiating `L` through it gives back exact per-particle marginal weights, for whichever latent(s) `J` was attached to.

How it fits together

The whole picture



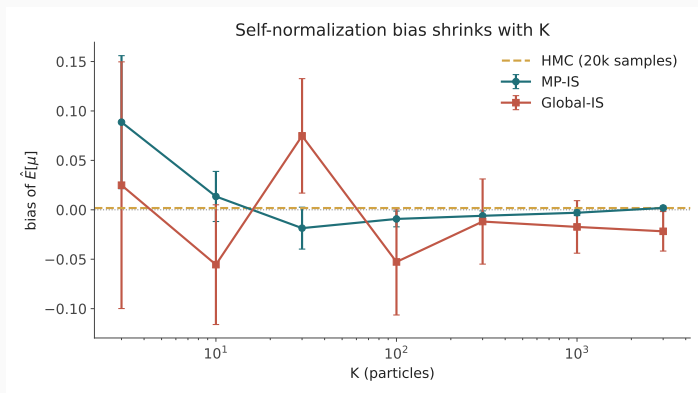
Newer empirical follow-ups

What I've been doing on top of this

All still grounded in the same $\hat{P}_{\text{MP}}(z)$ estimator and the source term trick above. Four quick results:

1. Does MP-IS actually beat plain self-normalized IS / catch up to HMC in practice?
2. A bug hunt that uncovered a real fact about how `alan`'s plate reduction works.
3. Where jackknife bias-correction does and doesn't help.
4. Two early prototypes: a general debiasing scheme (BR-SNIS), and exact inference via Particle Gibbs — both enabled directly by $\hat{P}_{\text{MP}}(z)$'s unbiasedness.

1: MP-IS vs. Global-IS vs. HMC



MP-IS's bias shrinks toward HMC's own (tiny) bias as K grows; Global-IS (whole-vector, no plate exploitation) is 5–10 $\times$ worse at matched K throughout.

2 & 3: a bug hunt, and where jackknife helps

- A hand-rolled reimplementation didn't match `alan`'s own output — traced to a genuine, previously undocumented fact: each plate element reduces its own K particles *independently* before summing across the plate (an extra Rao–Blackwellization the plate structure buys for free).
- Marginal ESS (via the source term trick's marginals) is bounded by K and uneven across plate replicates — up to $4\times$ spread at matched K .
- Delete-one jackknife removes most of the small- K bias for variables that reduce via a flat ratio, but **not** for variables reduced through a nested per-plate-element step — a precise, structural distinction, not a coincidence.

4: two prototypes enabled by $\hat{P}_{MP}$'s unbiasedness

BR-SNIS (chain-recycling debiasing):
only competitive with MP-IS's own
efficiency once pointed *at* the already
plate-reduced ratio — a flat version loses
badly.

Particle Gibbs / PGAS: pins a reference
trajectory using $\hat{P}_{MP}$'s unbiasedness for
exact posteriors. Ancestor sampling
(PGAS) gives a measured $74\times$ mixing
improvement over plain Particle Gibbs at
matched K .

Both are toy-model prototypes, not yet integrated into alan's core — write-ups and numbers
are in the design doc for whoever picks this up next.

Wrap-up

Summary

- TMC makes K -per-latent tractable by contracting along the plate graph.
- MP-RWS turns that estimator into a training algorithm, and proves it unbiased.
- The MP-IW bound is the resulting wake-phase objective — IWAE, scaled per latent instead of per whole vector.
- The source term trick gets samples/marginals/moments out of the *same* computation, via autodiff, for free.
- That unbiasedness is now being put to work: empirical bias checks, a real reduction-order finding, and two exact/debiasing prototypes building directly on it.

Questions